

## 1. Authentication

**Authentication** is the process of **verifying who the user is**.

- It answers the question: **“Who are you?”**
- Typically done using:
  - Username & Password
  - Tokens
  - Certificates
  - External providers (Google, Microsoft, etc.)

**In ASP.NET Core APIs:**

- Authentication happens **before** the request reaches the controller.
- If authentication fails → **401 Unauthorized** is returned.

### Example

User logs in with email and password → system verifies credentials → user is authenticated

---

## 2. Authorization

**Authorization** is the process of **verifying what the user is allowed to do**.

- It answers the question: **“What can you access?”**
- Happens **after authentication**
- Based on:
  - Roles (Admin, User)
  - Policies
  - Claims

**In ASP.NET Core APIs:**

- Controlled using attributes like:

[Authorize]

[Authorize(Roles = "Admin")]

## Example

User is authenticated ✓

User tries to access Admin API ✗ (if role is not Admin)

---

## 3. JWT Authentication

**JWT (JSON Web Token)** is a **stateless authentication mechanism** commonly used in REST APIs.

### What is JWT?

- A compact, URL-safe token
- Contains user information (claims)
- Signed to prevent tampering

### JWT Structure

Header.Payload.Signature

### Why JWT?

- Stateless (no server-side session storage)
- Scalable for microservices
- Ideal for REST APIs

### JWT Flow

1. User logs in with credentials
2. Server validates credentials
3. Server generates JWT
4. Client stores JWT (usually in header)
5. Client sends JWT with every request
6. Server validates JWT

### Authorization Header

Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...

---

## 4. How to Build ASP.NET Core REST API Using JWT Authentication (High-Level Steps)

### Step 1: Create ASP.NET Core Web API

```
dotnet new webapi
```

---

### Step 2: Install Required Packages

```
dotnet add package Microsoft.AspNetCore.Authentication.JwtBearer
```

---

### Step 3: Configure JWT Settings (appsettings.json)

```
"Jwt": {  
  "Key": "ThisIsASecretKey",  
  "Issuer": "MyApi",  
  "Audience": "MyApiUsers",  
  "ExpiryMinutes": 60  
}
```

---

### Step 4: Configure Authentication in Program.cs

```
buidler.Services.AddAuthentication("Bearer")  
    .AddJwtBearer(options =>  
    {  
        options.TokenValidationParameters = new TokenValidationParameters  
        {  
            ValidateIssuer = true,  
            ValidateAudience = true,  
            ValidateLifetime = true,  
            ValidateIssuerSigningKey = true,  
            ValidIssuer = builder.Configuration["Jwt:Issuer"],
```

```
ValidAudience = builder.Configuration["Jwt:Audience"],  
IssuerSigningKey = new SymmetricSecurityKey(  
    Encoding.UTF8.GetBytes(builder.Configuration["Jwt:Key"]))  
};  
});
```

```
builder.Services.AddAuthorization();  
app.UseAuthentication();  
app.UseAuthorization();
```

---

#### **Step 5: Create Login API to Generate JWT**

```
[HttpPost("login")]  
public IActionResult Login(LoginDto dto)  
{  
    // validate user (DB / hardcoded)  
  
    var claims = new[]  
    {  
        new Claim(ClaimTypes.Name, dto.Username),  
        new Claim(ClaimTypes.Role, "User")  
    };  
  
    var key = new SymmetricSecurityKey(  
        Encoding.UTF8.GetBytes(_config["Jwt:Key"]));  
  
    var token = new JwtSecurityToken(  
        issuer: "http://localhost:5000",  
        audience: "http://localhost:5000",  
        claims: claims,  
        signingCredentials: new SigningCredentials(key, SecurityAlgorithms.HmacSha256)  
    );  
    return Ok(new { token = token.ToString() });  
}
```

```
        issuer: _config["Jwt:Issuer"],
        audience: _config["Jwt:Audience"],
        claims: claims,
        expires: DateTime.UtcNow.AddMinutes(60),
        signingCredentials: new SigningCredentials(key, SecurityAlgorithms.HmacSha256)
    );

    return Ok(new
    {
        token = new JwtSecurityTokenHandler().WriteToken(token)
    });
}
```

---

## Step 6: Protect APIs Using Authorization

```
[Authorize]
[HttpGet("products")]
public IActionResult GetProducts()
{
    return Ok("Secure Data");
}
```

---

## Summary Table

| Concept        | Purpose                   |
|----------------|---------------------------|
| Authentication | Verifies user identity    |
| Authorization  | Verifies user permissions |

| Concept              | Purpose                              |
|----------------------|--------------------------------------|
| JWT                  | Stateless token-based authentication |
| ASP.NET Core JWT API | Secure REST API using tokens         |